POT JS

MANUAL

This is the Javascript API for the Go POT implementation that allows to write and read key-value pairs to and from the Swarm decentralized storage network.

A POT storage tree structure allows to have updateable key-value maps stored on an immutable, decentralized, content-addressable network like Swarm. It is called Proximity Order Tree (POT) after a fundamental, new storage technique that is described in a forthcoming paper.

INDEX

USE

QUICK START

- Drop `pot.wasm`, `potjs.js`, and `wasm_exec.js` in a directory.
- Write some html and js using functions **new()**, **put()**, **get()**, **save()**, **restore()**.
- Check out the function reference and examples.

There is no need to *build* the project, i.e., no need to install or run Go, to *use* it.

EXAMPLE

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

    await pot.ready()
    map = await pot.new()
    await map.put("hello", "POT!")
    value = await map.get("hello")
    console.log("key ‹hello› has:", value)
})()

</script>
```

Check out the function reference.

INTRODUCTION

This is an introduction to how POT JS is used. The basics are simple. You may work through the structured explanation below; or turn to the practical tutorial, to get a deeper understanding. You might decide to check out the separate function reference after a cursory look at this introduction, to beat your own path.

PURPOSE

The purpose of POT JS is to provide a simple and efficient way to store and retrieve key-value pairs to and from the immutable, decentralized storage network Swarm.

This is achieved with the main primitives being:

```
kvs = await pot.new(swarm, batch)
await kvs.put("some key", "some value")
value = await kvs.get("some key")
```

This almost constitutes a complete example program, cf. example 1 (pg. 30).

PROXIMITY ORDER TREE

POT JS allows to use proximity order trie (POT) functionality in the browser or to run it with node.js.

A proximity order tree is a data structure that allows to operate a key-value store on an immutable storage network. Concretely, it allows to use Swarm as a database; or to store a map-like structure to it, and be able to change it.

This evokes two oxymorons: an immutable storage would seem to not allow for any update of values, nor adding of additional key-value pairs. And Swarm is content-addressed: a hash of the content (not an arbitrary key) constitutes the address of the content in the network, to retrieve it by.

POT exploits a feature observed in Merkle tries when hashing its nodes: after an update, there is only one path back to the root that has to be re-hashed to arrive at a new root hash. All other hashes in the trie remain the same, as no values that are part of their pre-images have changed. POT uses this observation to implement an update to the data stored in the trie as a change of the root hash. Because a network like Swarm functionally provides the inverse to the hashing process – it returns the pre-image to the hash, as that is the definition of content-addressing – the update process of a Merkle trie can be repurposed as the re-indexing mechanism of a tree that stores content-addressed data. A new root hash will be created on every update that serves as the new entry point to the now-valid tree of data entries. Deleted nodes are not present anymore, updated ones in their new form. Those entries still exist in-storage. But they are not part of the hash tree anymore. The nodes that held the key-value pair of an updated pair that now has a new value are simply not part of the picture anymore as their combined key and value contents has changed its hash.

The keys to values are not a native search input to Swarm, which is made for retrieving pre-images of hashes. To search for a specific key to retrieve the value that was stored under it is therefore not something Swarm's base layer can help with. To navigate this data store at a useful pace – for the system to both understand where to find the value to a key, as well as where to insert the new value associated with a new key – a proximity order trie uses the bit patterns of a key's hash to establish a logarithmic time search order. It is described in a forthcoming paper.

POT JS uses the implementation of POT in Go:

https://github.com/ethersphere/proximity-order-trie

CODING

INITIALIZATION

For use in the browser, to initialize, include the generic **wasm_exec.js** and **potjs.js**, then wait for the initialization to finish:

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

   await pot.ready()

   ...
})()
...
```

DATA STORAGE

After **pot.ready()** resolves, the JS **pot** object can be used to create a new map. Use **put()** and **get()** functions on it.

This example uses an in-memory storage, the default that is useful for testing and prototyping.

```
map = await pot.new()

await pot.put("hello", "Hello, P.O.T.!")

value = await pot.get("hello")
```

See **example1.html**.

NETWORK CONNECTION

The connection to a Swarm network needs a URL to a Swarm bee node http API and a batch id to pay for storage. The arguments are given to the **new()** function.

For example, for node.js:

```
...
bee = process.argv[2]
batch = process.argv[3]

map = await pot.new(bee, batch)
...
```

An example invocation of these lines would be

```
% node hello.js \
http://localhost:1633 6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa
```

BATCH ID

A batch ID stands for 'postage stamps' that Swarm uses for incentivization, i.a. to compensate its Bee nodes to keep copies of the stored data.

The Makefile for example 9 is an example for how to start a local Swarm test network and create a batch id on it, in essence a free batch of stamps on your own local Swarm test network.

The following script extracts the new batch ID and stores it into .batch_id

```
@echo "⬡ starting five local swarm nodes"
fdp-play start --detach

@echo "⬡ buy stamps (free in this test setup)"
swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b | tee .batch_creation
grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
@echo "⬡ postage batch ID: $$(cat .batch_id)"

@echo "⬡ start tests (check browser)"
node test/test_node.js loc-net http://localhost:1633 $$(cat .batch_id)

@echo "⬡ wait before shutdown"
sleep 120

@echo "⬡ bee node logs"
docker container logs --tail 1000 fdp-play-queen

@echo "⬡ stopping nodes"
fdp-play stop
```

EXPLICIT INITIALIZATION

You may leave out **potjs.js** to initialize with more direct control: include only the generic Go WASM wrapper `wasm_exec.js`, create a `Go` object and instantiate the POT WASM object **pot.wasm**, then run it once it is loaded. See **example3.html**:

```
<script src="wasm_exec.js"></script>

<script>

const go = new Go()
WebAssembly.instantiateStreaming(fetch("pot.wasm"), go.importObject)
   .then((potwasm)=>{ go.run(potwasm.instance) })

...
```

Use onWasmLoaded() for the start of your POT JS use:

```
async function onWasmLoaded() {

   map = await pot.new()

   await map.put("hello", "POT!")

   value = await map.get("hello")

}
```

LOG LEVELS

The default log level is **DEBUG**, i.e., all log events are printed to browser log or screen.

0	**pot.NONE**	no logging
1	**pot.CRITICAL**	logs only errors that would appear to arise from a malfunction of POT JS.
2	**pot.ERROR**	programming and runtime errors are also logged.
3	**pot.INFO**	general runtime information is logged.
4	**pot.DEBUG**	specific data, like put and get keys and values are logged.

The verbosity can be adjusted with **setVerbosity()**, or – before the WASM executable is initialized, by setting **globalThis.potjs_verbosity** – The argument for the function can be an integer or a constant. Setting the **potjs_verbosity** variable, it must be a number.

Cf. examples/example8.js:

```
(async () => {
    await pot.ready()
    pot.setVerbosity(3)
    map = await pot.new()
})()
```

Note that the log prints to *screen* when running POT JS with node.js. In the browser, it logs into the *browser console*.

Because the default level is 4 = **DEBUG**, a production program will usually use **setVerbosity()** or **potjs_verbosity** to change that to a lower level.

Note that **potjs_verbosity** can only be used before the WASM executable is initialized and will not have any effect after. It is useful to suppress even the first two lines POT JS would otherwise log.

Cf. examples/examples7.js:

```
var go = new Go()
global.potjs_verbosity = 3
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)
    .then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {
    ...
```

ERROR STRATEGY

The error strategy of the API is JS-Style, using JS exceptions triggered from the go wasm functions via promise mechanisms, rather than returning error codes, Go-style.

The implementation of this is in Go though (`potjs.go`), not in JS, creating JS code in Go, with full data/situational access on the Go side. A major challenge is that Javascript exceptions cannot be thrown from Go. What works is to call the reject parameter of in the executor of JS promises that are constructed in Go. Because the most relevant functions are implemented like this at any rate, this covers most cases.

Synchronous functions, however, in part return JS error objects that have to be tested for where an exception would normally be expected.

CRASHES

The Go WASM executable, `pot.wasm`, can deadlock and crash, in which case it cannot be recovered. All POT JS functions stop working and throw an error instead that reports that the Go executable is down. In this case, `pot.wasm` must be re-initialized before the POT JS functions can be used again. This affects all `pot` and `map` functions, it means that while everything else might still work, literally the methods of a `map` that on the face look like native Javascript functions, are dead.

Reasons for crashing are deadlocks and uncaught panics on the side of the Go executable. Multiple measures are in place to keep the executable alive in all scenarios, to handle panics gracefully and to not allow a deadlock to happen. To maximize robustness, one might still want to program aware of this case, which means to avoid synchronous functions in production.

The capability to deadlock is not a fault of the Go POT implementation but presumably lies in the single-thread nature of Javascript that can be programmed highly asynchronously but is not truly parallel, multi-threaded, or multi-process (except when using the duly limited Web Workers). That Javascript is designed this way provides strong safeguards against a class of difficult to track errors. But they are coming off when crossing from Javascript into Go-land *and back*: a blocking wait in Go that hands control back to the browser deadlocks its main Javascript execution thread. Javascript waits for Go to continue and Go for Javascript. This is reliably the case when Go executes a network fetch that is handed back from WASM to Javascript to execute. When using POT JS to write to and read from Swarm, almost every function contains a fetch. When programming against one of the in-memory persistence models, none does.

FURTHER READING

Refer to the examples in the examples/ folder and or the Examples chapter to understand how POT JS might be used in the specific use case you envision.

The function reference in the doc/ folder explains all available functions with additional example snippets.

The tutorial visits a number of relevant angles to get a fuller picture.

TUTORIAL

This is a step by step guide how to use POT JS in the browser, or with node.js.

We'll discuss the building blocks, alternative options, and aspects that deserve attention.

We will start with a minimal in-browser example, develop it into a more interactive experience that is the prototype of a web app while shifting functionality from the browser to the server-side to show both options.

If you are really fast, frankly, checking out `examples/example1.html` may be all you need to get going, then use the `function reference` document in `doc/`.

WEB APP

INSTALLATION

For our first tutorial, create a new directory, and clone POT JS into it

```
% mkdir t1 ; cd t1
% git clone https://github.com/brainiac-five/potjs
```

There is no building or installation required beyond this.

The files we need will be `potjs/lib/pot.wasm` and `potjs/lib/wasm_exec.js`. That is, the Go implementation with JS interface compiled to WASM, and Go's standard JS WASM wrapper. They are both portable across operating systems and browsers.

PHASES

In general, there are the initialization, standard operations, and error conditions to think of. We will look at the happy path first.

INITIALIZATION

POT JS uses the Go implementation of POT. The Go parts are compiled to a WASM executable, which is loaded into Javascript. The procedure is the about the same in both the browser and node.js.

For our first example, create a html file **t1.html** that has the following contents:

```
<script src="potjs/lib/wasm_exec.js"></script>
<script>
  const go = new Go()
  WebAssembly.instantiateStreaming(fetch("potjs/lib/pot.wasm"), go.importObject)
     .then((r)=>{ go.run(r.instance) })
  async function onWasmLoaded() {
     console.log("READY.")
  }
</script>
```

tutorial/t1-step-1.html

This is the standard for initialization that any project using POT JS needs to employ. In node.js a different function is used for streaming, and it can all be used pre-packaged, as we will show later. But the basics to get started are always this:

The first line includes Go's standard JS WASM wrapper.

The fourth line **WebAssembly.instantiateStreaming(…)** loads the Go implementation of POT in its web assembly form, **pot.wasm**, to be used with Javascript.

Once it is loaded, it is executed by **go.run(r.instance)**, and actually does not stop. It keeps running and waiting for the next call.

But when its initializations are done – e.g., registering the Javascript function **pot.new()** etc. – it calls **onWasmLoaded()** to signal that the POT functionality is now usable.

To run the script, serve the directory with a web server, e.g. ..:

```
% npx http-server -c1 .
```

... and in a browser open:

```
http://localhost:8080/t1.html
```

The page will be blank. If you open the developer console (with Mac Chrome, option + command + J), you should see:

```
pot:  » POTWASM
pot:  » init done
t1.html:12 READY.
pot:  » ready
```

browser console of tutorial/t1-step-1.html

In the terminal you should have log entries by **http-server**:

```
Starting up http-server, serving .

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 1 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://127.0.0.1:8080
  http://192.168.178.192:8080
Hit CTRL-C to stop the server

[2025-09-13T11:45:40.420Z]  "GET /t1.html" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
(node:6393) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is deprecated
(Use `node --trace-deprecation ...` to show where the warning was created)
[2025-09-13T11:45:40.476Z]  "GET /potjs/lib/wasm_exec.js" "Mozilla/5.0 (Macintosh; Intel Mac
OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-09-13T11:45:40.567Z]  "GET /potjs/lib/pot.wasm" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```

terminal output of tutorial/t1-step-1.html

⟶ you successfully initialized POT.

STANDARD OPERATIONS

The main functions of POT JS are **put()** and **get()**. Maps are created with **new()**. They are saved to the network (or to in-memory POT if that is elected) using **save()** and retrieved with **newByReference()**.

We add a write and a read to our example. In **onWasmLoaded()** under **READY.** add:

```
map = await pot.new()
await map.put("hello", "POT!")
value = await map.get("hello")
console.log("key ‹hello› has:", value)
```

The first line creates a new key-value-store. What is special about POT is that it can be stored on an immutable storage network like Swarm. For our test though, we are using an in-memory storage for now.

The next line puts the string **"POT"** in the store, retrievable under the key **"hello"**.

The next line does exactly that: retrieve the value of key **"hello"**, which should be **"POT"**.

The last line prints the resulting **value** on the browser console, which will be **"POT"**.

To try it out, serve the directory, e.g., ...:

```
% npx http-server -c1 .
```

... and open the page:

```
http://localhost:8080/t1.html
```

The page will again be blank.

If you open the developer console, you should see:

```
pot:  » POTWASM
pot:  » init done
t1.html:12 READY.
pot:  » initialize new P.O.T.
pot:  › created new in-memory persister
pot:  › using in-memory persister
pot:  slot ref: 1
pot:  » ready
pot:  » put hello: POT!
pot:  » ⟶  68656c6c6f0000000000000000000000000000000000000000000000000000: 03504f5421
pot:  » get hello: POT!
pot:  » ⟵  68656c6c6f0000000000000000000000000000000000000000000000000000: 03504f5421
t1.html:20 key ‹hello› has: POT!
```

browser console of tutorial/t1-step-2.html

The middle part of the log after **READY.** is all about a new key-value-store being created. It is received in the variable **map** on the Javascript side.

The line **"put hello: POT!"** echoes the arguments that **kvs.put()** received.

Below, the digital values that are actually stored are shown.

`68656c6c6f000000000000000000000000000000000000000000000000000000` stands for the padded key **"hello"** and `03504f5421` for **"POT!"**. Keys are encoded differently than values. If the value to be stored was **"hello"**, instead of **"POT!"**, it would be `0368656c6c6f` instead of `03504f5421`.

The get log entry is showing the key AND the value the `get()` finds for it, not just its key parameter.

On the console, you should again see the logs of the three assets the http server provided:

```
[2025-09-13T12:21:26.674Z]  "GET /t1.html" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
(node:6487) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is deprecated
(Use `node --trace-deprecation ...` to show where the warning was created)
[2025-09-13T12:21:26.732Z]  "GET /potjs/lib/wasm_exec.js" "Mozilla/5.0 (Macintosh; Intel Mac
OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-09-13T12:21:26.819Z]  "GET /potjs/lib/pot.wasm" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```

part of the terminal output of tutorial/t1-step-2.html

⟶ you stored and retrieved a value from a POT.

PREMADE INITIALIZATION

You can include **potjs.js** to skip over having to type out the standard initialization. The easiest way to explain what that does is to show **potjs/lib/potjs.js**:

```
const go = new Go()

if(typeof pot == 'undefined') pot = {}

pot.start = new Promise((resolve, reject) => {
   let wasm = document.currentScript.getAttribute('wasm')
   if(!wasm) wasm = "pot.wasm"
   WebAssembly.instantiateStreaming(fetch(wasm), go.importObject)
          .then((r) => { go.run(r.instance) ; resolve(pot) })
          .catch((e) => { reject(e) })
})

pot.ready = () => { return pot.start }
```

lib/potjs.js

It wraps the initialization steps we discussed above into a promise and allows to inquire via `pot.ready()` whether the initialization has **started**. This is not quite as precise as waiting for the call of `onWasmLoaded()` and will be improved going forward because it can be too early.

However, using **potjs.js** our tutorial code can be abbreviated to:

```
<script src="potjs/lib/wasm_exec.js"></script>
<script src="potjs/lib/potjs.js" wasm="potjs/lib/pot.wasm"></script>

<script>

 async function onWasmLoaded() {

    console.log("READY.")

    map = await pot.new()

    await map.put("hello", "POT!")

    value = await map.get("hello")

    console.log("key ‹hello› has:", value)
 }

</script>
```

tutorial/t1-step-3.html

This code works exactly like the version before, giving the same logs.

SUB-COMPONENT INTEGRITY

To safeguard the integrity of the included scripts – but not the WASM script – fingerprints can be hardcoded into the HTML.

The required SHA-384 hashes can be found in potjs/lib/*.sha384. A "sha384-" must be prepended for the integrity tag.

The first two lines become:

```
<script src="potjs/lib/wasm_exec.js" integrity="sha384-
 PWCs+V4BDf9yY1yjkD/p+9xNEs4iEbuvq+HezAOJiY3XL5GI6VyJXMsvnjiwNbce"></script>
<script src="potjs/lib/potjs.js" wasm="potjs/lib/pot.wasm" integrity="sha384-
 woy0Fou09EcyFemKbNcnl+HI7XRxaENJQgJPdgPuBMqaqXwTWzUgF+zlViGzeQQF"></script>
```

beginning of tutorial/t1-step-4.html

This code works exactly like before, showing the same results.

ERROR CONDITIONS

The WASM executable is not a library but a Go program that runs continually, waiting for the next request. If it crashes, the functionality it extends just goes away. The main methods of JS objects that POT JS defines become unavailable.

It is relatively easy to deadlock the Go WASM by using the synchronous functions in the wrong context. Those are the POT JS functions ending on **Sync**. They are generally not safe to use when using a Swarm network as storage backend, that is, in the standard case.

In case the Go WASM program goes down, it has to be re-initialized.

NODE APP

For the next step, we will look at how POT JS works with node.js instead of the browser.

We'll create a single-script web app next that can persist data it handles server-side on Swarm. The result is going to be almost identical to example 8 (**examples/example8.js**).

Create a new folder, change into it and clone potjs into it.

```
% mkdir t2 ; cd t2
% git clone https://github.com/brainiac-five/potjs
```

INSTALLATION

For clarity, copy the two executables we need out of the repo and delete the rest.

```
% cp potjs/lib/pot.wasm .
% cp potjs/lib/wasm_exec.js .
% rm -rf potjs
```

Create a node.js/potjs test script **t2.js** and run it.

```
fs = require("fs");
require("./wasm_exec");

var go = new Go();

WebAssembly.instantiate(fs.readFileSync("./pot.wasm"), go.importObject)
      .then((r) => { go.run(r.instance) })
```

```
onWasmLoaded = async () => {

	map = await global.pot.new()
	await map.put("hello", "node")
	value = await
map.get("hello")
	console.log("hello:", value)
}
```

./tutorial/t2-step-1.js

The filesystem **fs** is required for **readFileSync()**, to read the wasm file.

The generic **wasm_exec** is also the same as for use in the browser.

As before, a **Go** object defined in **wasm_exec.js** is created; and when loading is done, the WASM executable is **run**.

Our program starts when **onWasmLoaded()** is called from **pot.wasm** as soon as it is done with initializations. A new **map** is created, to which a key and value are **put()**, then **get()** reads it back out.

Finally, the key, and value, as retrieved, are printed to the console.

Running it

```
% node t2.js
```

your should see:

```
pot:  » POTWASM
pot:  » init done
pot:  » initialize new P.O.T.
pot:  › created new in-memory persister
pot:  › using in-memory persister
pot:  slot ref: 1
pot:  » ready
pot:  » put hello: node
pot:  » ⟶  68656c6c6f0000000000000000000000000000000000000000000000000000:
  036e6f6465
pot:  » get hello: node
pot:  » ⟵  68656c6c6f0000000000000000000000000000000000000000000000000000:
  036e6f6465
hello: node
```

The **pot.wasm** library is started and logs the two first lines.

Then a new map is created, to which a key and value are written, then read back out.

The log shows the binary values that are written and read to the POT.

Eventually, the key and value are logged to the console.

Our starting point is thus basically **example/example7.js**.

HTTP SERVER

We add a minimal web server.

To the t2.js script append:

```
const http = require('http')
const page = “Hello, Swarm!”
const server = http.createServer(function(request, response) {

  response.writeHead(200, {'Content-Type': 'text/html'})
  response.end(page)
})
.listen(3000, '127.0.0.1')
```

part of ./tutorial/t2-step-2.js

Run it

```
% node t2.js
```

Open a browser and point to **http://localhost:3000**.

```
% open http://localhost:3000
```

It should show

```
Hello, Swarm!
```

INPUT FORM

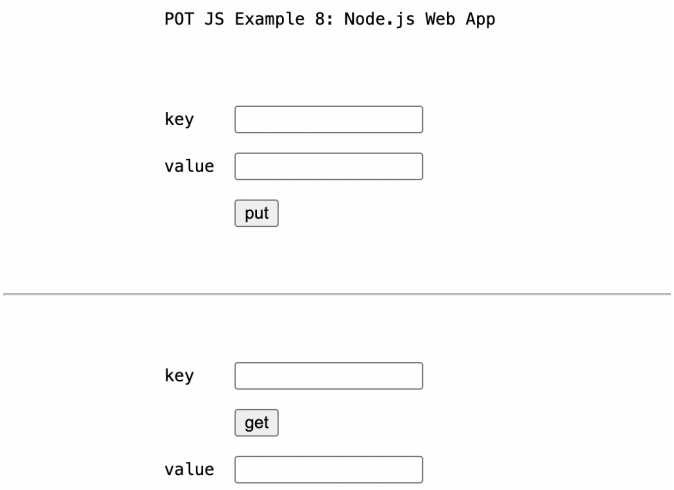
Stop the server and enhance the definition of the **page** variable to become an input form:

```
const page = `
<pre>

    <form method=post action="http://localhost:3000">

    key    <input name=key>

    value  <input name=value>
```

```
            <input type=submit name=button value=put>
  <hr />
     key    <input name=search>
            <input type=submit name=button value=get>
     value  <input name=result>
     </form>
  </pre>`
```

part of ./tutorial/t2-step-3.js

Running the server again, reloading the page **http://localhost:3000** in the browser you should see:



screen of ./tutorial/t2-step-3.js

INTERACTION

To interact with the form in the browser, we'll first clean up marginally. Change the WASM hook to:

```
global.onWasmLoaded = () => {
  kvs = pot.newSync()
}
```

part of ./tutorial/t2-step-4.js

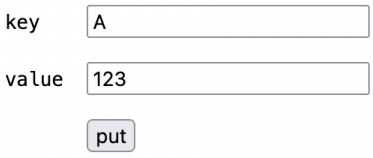
Replace the request handler, **server()**, with:

```
const http = require('http')
const qs = require('querystring')

const server = http.createServer(function(request, response) {

 if (request.method == 'POST') {

    var body = ''
    request.on('data', function(data) {
           body += data
    })

    request.on('end', async function() {

           var value = "put"
           post = qs.parse(body)

           if(post.button == 'put')
                  await kvs.put(post.key, post.value)
           else
                  value = await kvs.get(post.search)

           response.writeHead(200, {'Content-Type': 'text/html'})
           response.end(value)
    })
 }
 else {
    response.writeHead(200, {'Content-Type': 'text/html'})
    response.end(form)
 }
})
.listen(3000, '127.0.0.1')
```

The following line reads the value entered in the second form field and writes it to the POT. The first form field is used to enter the key that it is saved under.

```
                  await kvs.put(post.key, post.value)
```

In the browser, this looks like this: e.g., entering **A** for the key, and **123** for the value.



After clicking put, the browser then simply shows one word, '**put**'. Meanwhile, **kvs.put()** is triggered in the server and the value **123** is stored under key **A**.

Where it is actually stored is the **pot.wasm** executable that keeps running all the time while the server operates. It is stored as in-memory key-value-store for now, which is a good thing for testing.

What you should see now in the terminal is this:

```
pot:  » POTWASM
pot:  » init done
pot:  » initialize new P.O.T.
pot:  » created new in-memory persister
pot:  › using in-memory persister
pot:  slot ref: 1
pot:  » ready
pot:  » put A: 123
pot:  › ⟶  41000000000000000000000000000000000000000000000000000000000000000000:
  03313233
```

To get back to the form, use the browser's back button.

If you enter `A` in the third form field, and click get, the browser will show '`123`.`

The terminal will add:

```
pot:  » get A: 123
pot:  › ⟵  41000000000000000000000000000000000000000000000000000000000000000000:
  03313233
```

This is a round-trip, using our app.

VERBOSITY

The logging to screen is relatively detailed. Maybe you want to see less to have a better overview, or not have anything at all because you want to log different things to the console or just keep it clean without diverting output.

The level of logging can be changed with a value from 0 to 4, zero meaning no logging at all, 4 what you saw in the terminal up to now. The levels have names:

0	`pot.NONE`	no logging, not even errors.
1	`pot.CRITICAL`	logs only errors that appear to arise from a POT JS malfunction.
2	`pot.ERROR`	programming and runtime errors are also logged.
3	`pot.INFO`	general runtime information is logged.
4	`pot.DEBUG`	specific data, like put and get keys and values are logged.

We can set this verbosity level using `pot.setVerbosity(<level>)`. In our program, we can amend the `onWasmLoaded()` function to:

```
global.onWasmLoaded = () => {

  pot.setVerbosity(pot.INFO)

  kvs = pot.newSync()
}
```

part of ./tutorial/t2-step-5.js

Making the same put and get test as before, the lowest level log lines are gone:

```
pot:  » POTWASM
pot:  » init done
pot:  » initialize new P.O.T.
pot:  » created new in-memory persister
pot:  » ready
pot:  » put A: 123
pot:  » get A: 123
```

Setting verbosity to pot.NONE results in this output. Two lines are left, even with pot.NONE.

```
pot:  » POTWASM
pot:  » init done
```

To also eliminate the remaining first two lines, a different method has the be used. The global variable potjs_verbosity needs to be set before WASM is initialized, e.g., like this:

```
var go = new Go()

globalThis.potjs_verbosity = 0

WebAssembly.instantiate(fs.readFileSync("./pot.wasm"), go.importObject)

      .then((r) => { go.run(r.instance) })


global.onWasmLoaded = () => {
    kvs = pot.newSync()
}
```

part of ./tutorial/t2-step-6.js

This suppresses even the first two lines coming from POT JS.

The value set to **globalThis.potjs_verbosity** must be a number, a literal or a variable you defined but **pot.NONE** etc. do not exist yet at this point. The initialization of WASM creates them.

The value in **globalThis.potjs_verbosity** matters at the point when the pot.wasm is initialized and changing it has no effect after that point. The function **pot.setVerbosity()** can be changed without immediate effect, any time.

SINGLE-PAGE

We'll add a little more polish by staying on the page.

After the **onWasmLoaded()** function, insert two functions, **put()** and **get()**:

```
async function put(key, value) {

  await kvs.put(key, value)

  return `put ${key}: ${value}`
}

async function get(search) {

  value = await kvs.get(search)

  return `get ${search}: ${value}
     <script>
     document.getElementsByName('result')[0].value = '${value}'
     </script>`
}
```

part of ./tutorial/t2-step-5.js

Then replace the request hook with calls to the **put()** and **post()** functions we just defined.

They are replacing the direct calls to **kvs.put()** and **kvs.get()**.

We are also adding a variable, **log**, to gather a minimal status report.

```
    request.on('end', async function() {

            var log
            const post = qs.parse(body)

            if(post.button == 'put')
                    log = await put(post.key, post.value)
            else
                    log = await get(post.search)

            console.log("srv: " + log.match(/.*/)[0])

            response.writeHead(200, {'Content-Type': 'text/html'})
            response.end(form + '<hr><br><pre>\t\t' + log)
    })
```

part of ./tutorial/t2-step-5.js

The complete code we arrived at is:

```
const page = `<pre>

    <form method=post action="http://localhost:3000">

    key    <input name=key>

    value  <input name=value>

           <input type=submit name=button value=put>

 <hr />

    key    <input name=search>

           <input type=submit name=button value=get>

    value  <input name=result>

    </form>

 </pre>`

const fs = require("fs")
require("./wasm_exec") // note the ./

var go = new Go()
var kvs

WebAssembly.instantiate(fs.readFileSync("./pot.wasm"), go.importObject)
 .then((r) => { go.run(r.instance) })

global.onWasmLoaded = () => {

 kvs = pot.newSync()
}

async function put(key, value) {

 await kvs.put(key, value)

 return `put ${key}: ${value}`
}

async function get(search) {

 value = await kvs.get(search)

 return `get ${search}: ${value}
    <script>
    document.getElementsByName('result')[0].value = '${value}'
    </script>`
}

const http = require('http')
const qs = require('querystring')

const server = http.createServer(function(request, response) {

 if (request.method == 'POST') {

    var body = ''
    request.on('data', function(data) {
           body += data
    })

    request.on('end', async function() {
```

```
            var log
            const post = qs.parse(body)

            if(post.button == 'put')
                   log = await put(post.key, post.value)
            else
                   log = await get(post.search)

            console.log("srv: " + log.match(/.*/)[0])

            response.writeHead(200, {'Content-Type': 'text/html'})
            response.end(page + '<hr><br><pre>\t\t' + log)
      })
   }
   else {
      response.writeHead(200, {'Content-Type': 'text/html'})
      response.end(page)
   }
})
.listen(3000, '127.0.0.1')
```

./tutorial/t2-step-5.js

Running it

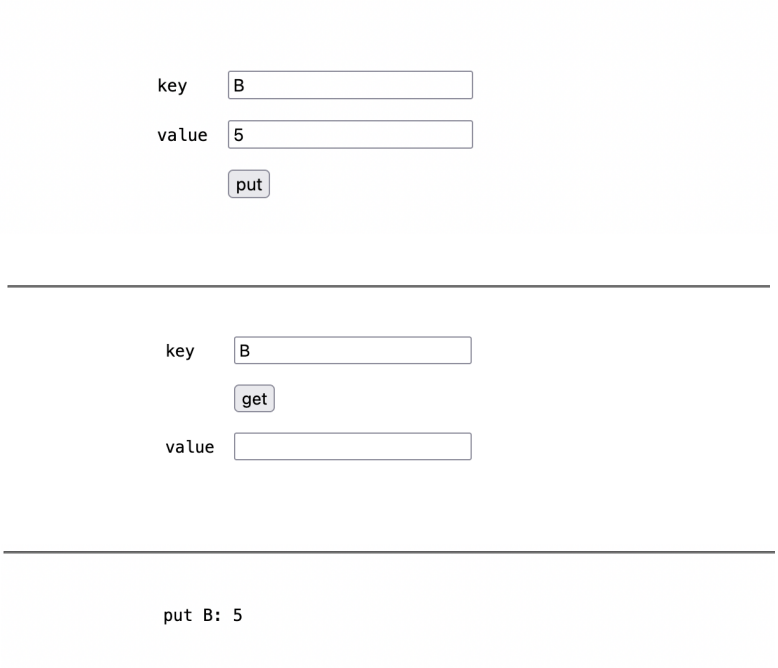
```
% node t2.js
```

We see the form fields as before, but pressing buttons does make the page disappear.

The result of the get is shown in the fourth form field.

Both put and get results are shown in the bottom line.

This is what you will see, although not at the same time:



screens of ./tutorial/t2-step-5.js

CONCLUSION

POTJS works with node.js as easily as it works in the browser. It doesn't get in the way and is a simple way to store state of a web app to a trustless, decentralized network.

FUNCTIONS

The focus of the functions are **get()** and **put()**. They are split in two by two main groups, for four different ways of calling. Additionally, there are init, test and support functions.

The groups are asynchronous / synchronous and typed / raw respectively. Asynchronous functions return promises, synchronous functions block and return a value, error or null. Typed functions mark the Javascript type in the storage, to automatically return the right type on retrieval. This can be boolean, number,[1] or string. Raw functions accept a Uint8Array of bytes to store and the right function has to be called to get the expected type back out. The synchronous functions' names end on Sync. The raw functions have Raw as a name component. The most relevant functions are put() and get(), which are asynchronous, typed functions. That means, get() returns a promise for the value to be retrieved and it will automatically be in the type that it was stored with put().

Synchronous functions can be handy for prototyping and special situations but generally only work for tests in the browser using the in-memory persistence of the Go POT implementation, not when connecting to Swarm or a test network, and not when using node. It is possible that they might work well with the background threads of Web Workers but this has not been tested. Therefore, they should simply NOT be used in production. The asynchronous functions work in all circumstances.

Raw functions will be useful in special situations, whenever binary data is handled, but usually the vanilla (typed, asynchronous) functions **new**, **save**, **put** and **get** will be the first choice.

For details and examples, see file **Pot – (III) 31 August 2025 function reference.pdf.**

MAP INITIALIZATION

pot.new	create a new Swarm key-value-store, i.e., a map, async
pot.newByReference	create a new Swarm map from a reference of a prior save, async
map.save	store a Swarm KVS to the network, async
pot.newSync	create a new Swarm key-value-store, i.e., a map, sync
pot.newByReferenceSync	create a new Swarm map from a reference of a prior save, sync
map.saveSync	store a Swarm KVS to the network, sync

TYPE-AWARE

put	return a promise for null or Error. Asynchronous.
get	return a promise for the typed get, the type will be as put. Async.
putSync	return a promise for null or Error. Synchronous.
getSync	return a promise for the typed get, i.e., the type will be as put. Sync.

[1] Javascript does not have native integers and stores all numbers as IEEE 754 64-bit floats internally.

RAW BYTE ACCESS

putRaw	put a raw **Uint8Bytes** array, async.
getRaw	get a raw **Uint8Bytes** array, async.
getBoolean	get the raw value as Boolean. First byte 0 for false, all else true, async.
getNumber	get the raw value as floating point number. Must be 8 bytes. Async.
getString	get the raw value as string, async.
putRawSync	put a raw **Uint8Bytes** array, synchronous.
getRawSync	get a raw **Uint8Bytes** array, synchronous.
getBooleanSync	get the raw value as Boolean. First byte 0 for false, all else true, sync.
getNumberSync	get the raw value as floating point number. Must be 8 bytes. Sync.
getStringSync	get the raw value as string, synchronous.

POT INITIALIZATION

pot.ready	promise that resolves when pot.wasm is initialized. Include **potjs.js**.
onWasmLoaded	called, if it exists, by pot.wasm when it is initialized. Without **potjs.js**.
pot.setOptimization	controls the extent to which POT JS writes to the log, a value from 0 to 5.
globalThis.potjs_optimization	setting optimization before initialization
pot.setVerbosity	controls the extent to which POT JS writes to the log, a value from 0 to 5.
globalThis.potjs_verbosity	setting verbosity before initialization

TESTING AND SUPPORT

These functions are not used in production. They serve to test the integrity of POT JS.

log	write to browser console via Go to verify a language-land roundtrip
hello	write hello to browser console to verify established WASM stream
randKey	32 random bytes for key testing
randValue	79 to 101 random bytes for value testing (range from Go POT)
type_encoded_bytes	JS type detection and value encoding with correct leading type byte
type_decoded_value	JS type detection of raw kvs value by leading type byte
hangingPromise	blocking promise (in Go) with time out for exception testing
setFail	trigger the next mock storage function called to return an error
setPanic	trigger the next mock storage function called to raise a panic
setDelay	trigger the next mock storage function called to stall for a given time
setHang	trigger the next mock storage function called to stall indefinitely

EXAMPLES

To run the examples, serve the main directory with `npx http-server .` and open `http://127.0.0.1:8080/example1.html`.

Or use

```
% make example1
```

etc. There are nine examples 1-9 that can be started this way.

`example1.html`	briefest example as listed above, in-memory storage
`example2.html`	interactive example, input fields, integrity
`example3.html`	example 2 dropping potjs.js for more control
`example4.html`	example 1 using synchronous access functions
`example5.html`	example 1 but with connection to a storage network
`example6.html`	example 5 with better connection to make rules
`example7.js`	example 1 for execution with node.js
`example8.js`	Single-source web app with node.js
`example9.js`	Identical to example 8, just called differently

EXAMPLE 1

A simple put and get, logging to console.

`example1.html`

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

  await pot.ready()
  map = await pot.new()
  await map.put("hello", "POT!")
  value = await map.get("hello")

  console.log("key ‹hello› has:", value)
})()

</script>
```

WHAT YOU SHOULD SEE

BROWSER CONSOLE:

```
pot:  » POTWASM
wasm_exec.js:22 pot:  » init done
wasm_exec.js:22 pot:  » ready
wasm_exec.js:22 pot:  » initialize new P.O.T.
wasm_exec.js:22 pot:  slot ref: 1
wasm_exec.js:22 pot:  » put hello: POT!
wasm_exec.js:22 pot:  » ⟶  68656c6c6f0000000000000000000000000000000000000000000000000000000000000000:
03504f5421
wasm_exec.js:22 pot:  » get hello: POT!
wasm_exec.js:22 pot:  » ⟵  68656c6c6f0000000000000000000000000000000000000000000000000000000000000000:
03504f5421
example1.html:29 key ‹hello› has: POT!
```

EXAMPLE 2: INTERACTION

Interactive use and integrity checks.

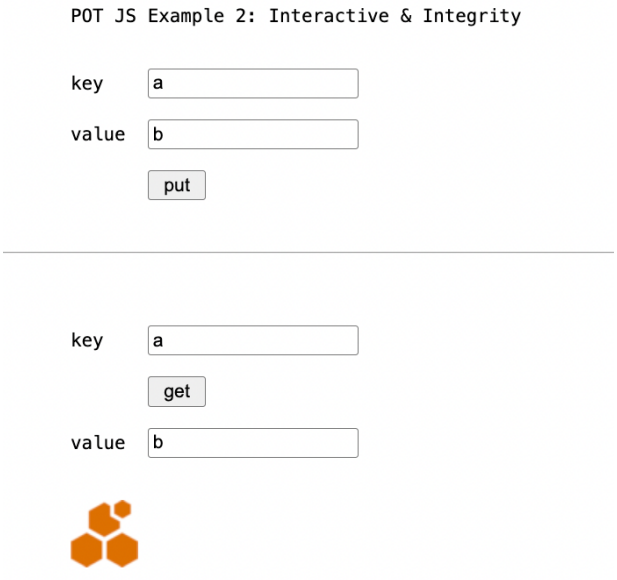
example2.html

```
<pre>

 POT JS Example 2: Interactive & Integrity

 key    <input id=key>
 value  <input id=value>
        <button onclick="put()"> put </button>

<hr />

 key    <input id=search>
        <button onclick="get()"> get </button>
 value  <input id=result>

 <img src=favicon.ico>

</pre>
```

```
<script src="wasm_exec.js" integrity="sha384-
 PWCs+V4BDf9yY1yjkD/p+9xNEs4iEbuvq+HezAOJiY3XL5GI6VyJXMsvnjiwNbce"></script>
<script src="potjs.js" integrity="sha384-
 dUfhLIycF6GUprYxIlIiAAQR3rh3KsPxPdsgq4tP2rTe7grND7bXjAj4nawjq5Ht"></script>

<script>

 var map

 (async () => {

    await pot.ready()
    map = await pot.new()

 })()

 const field = (id) => { return document.getElementById(id) }

 async function put() {

    key = field('key').value
    value = field('value').value
    await map.put(key, value)
 }

 async function get() {

    key = field('search').value
    value = await map.get(key)
    field('result').value = value
 }

</script>
```

WHAT YOU SHOULD SEE

After entering a value for key and value, and hitting put. Then entering the same key in the lower screen area and hitting get should yield the value in the lowest field.



BROWSER CONSOLE:

```
pot:  » POTWASM
wasm_exec.js:22 pot:  » init done
wasm_exec.js:22 pot:  » ready
wasm_exec.js:22 pot:  » initialize new P.O.T.
wasm_exec.js:22 pot:  slot ref: 1
wasm_exec.js:22 pot:  » put a: b
wasm_exec.js:22 pot:  » →  61000000000000000000000000000000000000000000000000000000000000000000:
0362
wasm_exec.js:22 pot:  » get a: b
wasm_exec.js:22 pot:  » ←  61000000000000000000000000000000000000000000000000000000000000000000:
0362
```

EXAMPLE 3: INITIALIZATION

More explicit initialization.

This page does not use pot.js but initializes pot.wasm directly.

example3.html

```
<pre>

	POT JS Example 3: Explicit Initialization

	key	<input id=key>
	value	<input id=value>
		<button onclick="put()"> put </button>

<hr />

	key	<input id=search>
		<button onclick="get()"> get </button>
	value	<input id=result>

</pre>
<script src="wasm_exec.js"></script>
<script>
	const go = new Go()
	var map;
```

```
	WebAssembly.instantiateStreaming(fetch("pot.wasm"), go.importObject)
		.then((r)=>{
			go.run(r.instance)
			map = pot.newSync()
		})

	const field = (id) => { return document.getElementById(id) }

	async function put() {

		key = field('key').value
		value = field('value').value
		await map.putSync(key, value)
	}

	async function get() {

		key = field('search').value
		value = await map.getSync(key)
		field('result').value = value
	}
</script>
```

EXAMPLE 4: SYNCHRONOUS

This page uses synchronous calls that can be helpful for prototyping but don't work for network connections (only in-memory = new() without parameters) and not for use with node.

example4.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

function onWasmLoaded() {

	map = pot.newSync()

	map.putSync("hello", "POT!")

	value = map.getSync("hello")

	console.log("key ‹hello› has:", value)
}

</script>
```

EXAMPLE 5: NETWORK

This page connects to a local Swarm network.

The example is for illustration, the batch id would have to be updated. See example5.html (and 6).

example5.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>
(async () => {
      await pot.ready()

      map = await pot.new("http://localhost:1633",
      "6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa")

      await map.put("hello", "POT!")

      value = await map.get("hello")

      console.log("key ‹hello› has:", value)

      ref = await map.save()

      console.log("tree root is:", ref)
})()
</script>
```

EXAMPLE 6: NETWORK II

Example 6 works like example 5 but can be run directly with `make example6`. It has parameters

See **example6.html**

```
...
        await pot.ready()

        batch = new URLSearchParams(window.location.search).get('batch')

        map = await pot.new("http://localhost:1633", batch)

        await map.put("hello", "POT!")
...
```

EXAMPLE 7: NODE

This example shows how POT JS works with node.js.

It also demonstrates how logging verbosity can be changed. Setting potjs_verbosity to 0 suppresses all logging from POT JS.

example7.js

.

```
fs = require("fs");
require("./wasm_exec"); // note the ./

var go = new Go();

global.potjs_verbosity = 3

WebAssembly.instantiate(fs.readFileSync("pot.wasm"), go.importObject)
	.then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {

	console.log("wasm loaded-function called")

	map = await global.pot.new()

	await map.put("hello", "node")

	value = await map.get("hello")

	console.log("hello:", value)
}
```

WHAT YOU SHOULD SEE

TERMINAL:

```
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go nomock.go
● √ pot.wasm built for production
node examples/example7.js


	POT JS Example 7: Node

	Check out the source in example7.js.


pot:  » POTWASM
pot:  » init done
wasm loaded-function called
pot:  » initialize new P.O.T.
```

```
pot:  › created new in-memory persister
pot:  › using in-memory persister
pot:  » ready
pot:  » put  hello: node
pot:  » get  hello: node
hello: node
```

EXAMPLES 8 AND 9: WEB APP

These examples demonstrate a single-source web app. The page is served from the server script. The POT functions run on the node.js server.

Examples 8 and 9 are identical to make a point: that the storage destination for the POT can be switched out by parameters. In this case, from in-memory storage (8) to local network (9).

example8.js and example9.js

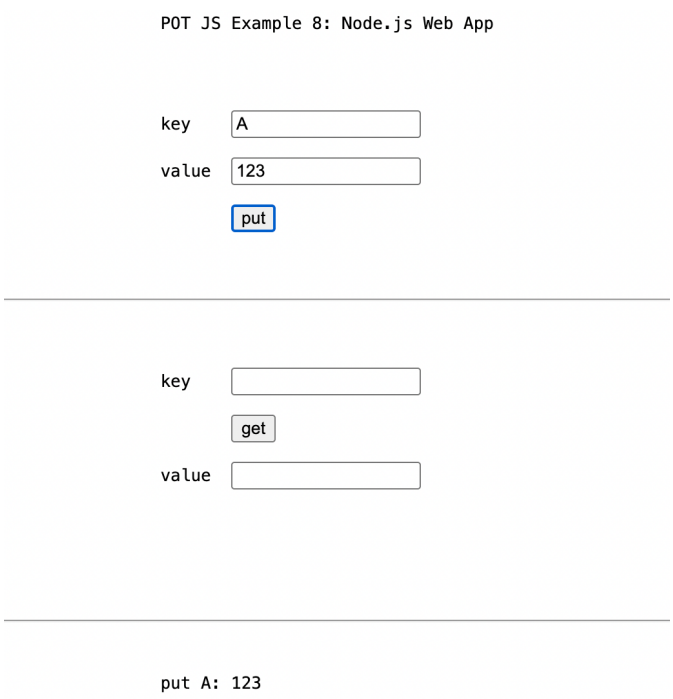
```
console.log(`
      POT JS Example 8: Node.js Web App Server
      This is a self-contained web app, serving
      a page to receive input, returning results.
`)

const http = require('http')
const qs = require('querystring')
const form = `<pre>

            POT JS Example 8: Node.js Web App
            <form method=post action="http://localhost:3000">
            key    <input name=key>
            value  <input name=value>
                   <input type=submit name=button value=put>
      <hr />
            key    <input name=search>
                   <input type=submit name=button value=get>
            value  <input name=result>
            </form>
      </pre>
```

```
	<script> console.log('see server log') </script>`

const fs = require("fs")
require("./lib/wasm_exec") // note the ./

var go = new Go()
var kvs

WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)
	.then((r) => { go.run(r.instance) })
global.onWasmLoaded = () => {

	bee = process.argv[2]
	batch = process.argv[3]
	console.log(bee, batch)

	kvs = pot.newSync(bee, batch)
	console.log('srv: KVS initialized')

	// Notes:
	// * The "pot: " log entries are coming from Go pot.wasm.
	// * There is no catching of early calls of put() and get()
	// * onWasmLoaded could be defined async, for await pot.new()
	// * onWasmLoaded has to be added to global to get detected
}

async function put(key, value) {

	await kvs.put(key, value)

	return `put ${key}: ${value}`
}

async function get(search) {

	value = await kvs.get(search)

	return `get ${search}: ${value}
		<script>
		document.getElementsByName('result')[0].value = '${value}'
		</script>`
}

const server = http.createServer(function(request, response) {
	if (request.method == 'POST') {

		var body = ''
		request.on('data', function(data) {
			body += data
		})

		request.on('end', async function() {

			var log
			const post = qs.parse(body)

			if(post.button == 'put')
				log = await put(post.key, post.value)
			else
				log = await get(post.search)

			console.log("srv: " + log.match(/.*/)[0])

			response.writeHead(200, {'Content-Type': 'text/html'})
			response.end(form + '<hr><br><pre>\t\t' + log)
```

```
            })
      }
      else {
            response.writeHead(200, {'Content-Type': 'text/html'})
            response.end(form)
      }
})
.listen(3000, '127.0.0.1')

console.log(`srv: listening at http://127.0.0.1:3000`)
```

WHAT YOU SHOULD SEE

BROWSER:



The screen will refresh with fields empty on every button press.

TERMINAL:

```
% make example8
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go nomock.go
⬢ √ pot.wasm built for production
open http://127.0.0.1:3000/examples/example8.html
node examples/example8.js
```

```
       POT JS Example 8: Node.js Web App Server

       This is a self-contained web app, serving
       a page to receive input, returning results.

srv: listening at http://127.0.0.1:3000
pot:  » POTWASM
pot:  » init done
pot:  » initialize new P.O.T.
pot:  › created new in-memory persister
pot:  › using in-memory persister
pot:  slot ref: 1
srv: KVS initialized
pot:  » ready
pot:  » put A: 123
pot:  » ⟶  4100000000000000000000000000000000000000000000000000000000000000: 03313233
srv: put A: 123
pot:  » get A: 123
pot:  » ⟵  4100000000000000000000000000000000000000000000000000000000000000: 03313233
srv: get A: 123
pot:  » put A: 123
pot:  » ⟶  4100000000000000000000000000000000000000000000000000000000000000: 03313233
srv: put A: 123Node
```

BUILDING

There is **no need for building POT JS to use it.**

The main executable, the Go WASM file **pot.wasm,** and the auxiliary Javascript and HTML files are portable.

To build, run

```
% GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
% cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" .
```

or use

```
% make build
```

WHAT YOU SHOULD SEE

```
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
√ created pot.wasm for production
```

CLEAN BUILD

To rebuild from scratch, including integrity hashes and rebuilding **go.mod**, use:

```
% make clean build
```

WHAT YOU SHOULD SEE

```
rm -f go.mod
rm -f pot.wasm
rm -f wasm_exec.js
rm -f potjs.js.sha384 wasm_exec.js.sha384
rm -f .batch_creation .batch_id
go mod init potwasm
go: creating new go.mod: module potwasm
go: to add module requirements and sums:
    go mod tidy
go mod edit -replace github.com/ethersphere/proximity-order-
trie=github.com/ethersphere/proximity-order-trie@v1.0.0
go get
go: added github.com/beorn7/perks v1.0.1
go: added github.com/cespare/xxhash/v2 v2.2.0
go: added github.com/ethersphere/bee/v2 v2.5.0
go: added github.com/ethersphere/proximity-order-trie v1.0.0
go: added github.com/hashicorp/errwrap v1.0.0
go: added github.com/hashicorp/go-multierror v1.1.1
go: added github.com/prometheus/client_golang v1.18.0
go: added github.com/prometheus/client_model v0.6.0
go: added github.com/prometheus/common v0.47.0
go: added github.com/prometheus/procfs v0.12.0
go: added golang.org/x/crypto v0.23.0
go: added golang.org/x/sys v0.20.0
go: added google.golang.org/protobuf v1.33.0
cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" .
openssl dgst -sha384 -binary potjs.js | openssl base64 -A > potjs.js.sha384
openssl dgst -sha384 -binary wasm_exec.js | openssl base64 -A >
wasm_exec.js.sha384
for fn in potjs.js.sha384 wasm_exec.js.sha384; do sed -i.bak -e "s/\(\"${fn:0:-
7}\" integrity=\)[^>]*/\1\"sha384-${$(cat ${fn})//[\/]/\\/}\"/"
example1.html ; rm -f example1.html.bak ; done
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
√ created pot.wasm for production
```

REQUIREMENTS

POT JS includes the Go implementation of POT.

POT JS API

Tested with:

- go 1.24.0
- node.js 22.17.1
- chrome 138.0.7204.184 arm64

probably works with earlier versions.

GO POT

- go 1.24.0

- testify 1.8.4
- swarm bee 2.5.0
- crypto 0.23.0
- sync 0.14.0

Which will require

- perks 1.0.1
- xxhash 2.2.0
- go-spew 1.1.1
- errwrap 1.0.0
- go-multierror 1.1.1
- text v0.2.0
- go-difflib 1.0.0
- prometheus
- x/sys 0.20.0
- protobuf 1.33.0
- yaml 3.0.1

See `go.mod`.

TESTS

**To test the JS API, serve the main directory, e.g., with with npx http-server .
and open http://127.0.0.1:8080/test.html?tag=in-mem.**

Or use

```
% make test
```

It uses `test.js` and `test.css` to display the results from a suite of 142 tests across the available **put** and **get** functions, and more.

WHAT YOU SHOULD SEE

TERMINAL

```
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
√ created pot.wasm for production
⬡ start http server (stop with 'make stop')
(npx http-server -c1 . &)
open "http://127.0.0.1:8080/test.html?tag=in-mem"
Starting up http-server, serving .

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 1 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://127.0.0.1:8080
  http://192.168.178.192:8080
Hit CTRL-C to stop the server

[2025-08-30T21:44:02.883Z]  "GET /test.html?tag=in-mem" "Mozilla/5.0 (Macintosh; Intel
Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
(node:22323) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is
deprecated
```

```
(Use `node --trace-deprecation ...` to show where the warning was created)
[2025-08-30T21:44:02.904Z]  "GET /test.css" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.906Z]  "GET /wasm_exec.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.910Z]  "GET /favicon.ico" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.989Z]  "GET /test.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.990Z]  "GET /kvs_test.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.997Z]  "GET /pot.wasm" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```

BROWSER

BROWSER CONSOLE

```
...

wasm_exec.js:22 pot:  ✦ case #140 » Cancel Delay
wasm_exec.js:22 pot:  <object>
wasm_exec.js:22 pot:  create hanging test promise and cancel it (delay, chain .catch)
wasm_exec.js:22 pot:  sleeping
wasm_exec.js:22 pot:  canceled!
wasm_exec.js:22 pot:  √ expected error: ‹Error: canceled›
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ✦ case #141 » Immediate Cancel II
wasm_exec.js:22 pot:  <object>
wasm_exec.js:22 pot:  create hanging test promise and cancel it immediately (chain
.catch)
wasm_exec.js:22 pot:  sleeping
wasm_exec.js:22 pot:  canceled!
wasm_exec.js:22 pot:  √ expected error: ‹Error: canceled›
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ••• Test Suite #14 ••• FAILURES •••▶IN-MEM◀
wasm_exec.js:22 pot:  Failure tests are available in extended API test mode. Run `$make
xt`.
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ••• Test Suite #15 ••• PROOFS •••▶IN-MEM◀
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ✦ case #142 » Return a Proof
wasm_exec.js:22 pot:  <object>
wasm_exec.js:22 pot:  • new map
wasm_exec.js:22 pot:  » initialize new P.O.T.
wasm_exec.js:22 pot:  slot ref: 101
wasm_exec.js:22 pot:  √ no error raised.
wasm_exec.js:22 pot:  • get proof for K1
wasm_exec.js:22 pot:  √ as expected, ‹null›.
wasm_exec.js:22 pot:
wasm_exec.js:22 pot:  ••• tests cases run: 142 • assertions: 1325  •••
wasm_exec.js:22 pot:  ▶ no errors ◀
```

The standard tests run against the Go implementation of POT, like the use would be in production. The **extended tests**, however, focus on the JS AP to Go, add cancellation and time out cases. They do not interface with the Go POT code.

Latency emulation, promises, and connectivity as well as retention error conditions will be added to emulate the reality of an unreliable network as ultimate storage layer. Run the tests with `make test` and `make xt` respectively.

Note that `make xt` (specifically, `make mock` change the project configuration and build executables (especially a `pot.wasm`) that will not work for production. (They will appear to work but they sidestep Go POT.) Use `make unmock`, `make test` or `make build` to restore to the production target (i.e., build the real `pot.wasm`).

The **extended tests** use a separate, dedicated, in-memory mock storage to separate them from the Go POT implementation.

The tests are made for the browser as the final destination of the combined architecture, in either case (standard and extended tests) they are invoked by opening the file `test.html`. The very small file that is altered between the test modes is `testmode.js`, and `go.mod` to replace package the real Go POT with `mock/`

This test page and test framework is designed to bridge the language divide and allow for the emulation of exceptions in one degree of separation (caused in Go, caught from JS). The major relevance of the test harness is to help surface any friction between JS's and Go's resource release mechanisms (JS promises and Go channels) to detect possible leaks. Go's channels are not without peril – they can deadlock – and the challenge is aggravated by their position behind the JS API. The relevant code triggering the exceptional conditions resides in `mock.go`. For its purposes, it has to be part of the `main` package rather than package `pot` in `mock/`.

DEVELOPING

This chapter is about contributing to or changing POT JS itself.

Developing and testing at this point have been done on Mac only.

FILES

```
README.MD               subset of these instructions

potjs.go                Go JS interface
nomock.go               No-op version of test functions
mock.go                 monkey patch mock of Go POT for extended testing

mock/                   replacement of Go POT for extended testing
 pot.go
 go.mod
 pkg/persister          copy of Go POT in-memory storage to satisfy depencies

lib/
 pot.wasm               Go POT implementation compiled to WASM
 wasm_exec.js           generic Go WASM wrapper (replace w/your Go version's)
 potjs.js               Thin JS initialization convenience code
 wasm_exec.js.sha384    SHA384 checksum for sub-component verification
 potjs.js.sha384        SHA384 checksum for sub-component verification

doc/
 POT JS manual.odt
 POT JS function reference.pdf
 POT JS manual.pdf
 POT JS function reference.odt

examples/
 example1.html          briefest example as listed above, in-memory storage
 example2.html          interactive example, input fields, integrity
 example3.html          example 2 dropping potjs.js for more control
 example4.html          example 1 using synchronous access functions
 example5.html          example 1 but with connection to a storage network
 example6.html          example 5 with better connection to make rules
 example7.js            example 1 for execution with node.js
 example8.js            single-script node.js app, server and web page
 example9.js            example 8 verbatim stared with different arguments
 favicon.ico            Swarm logo to suppress browser errors

test/
 test.html              test main page for test, ext_test, locnet_test
 kvs_test.js            test suites
 test.js                generic test harness
 test.css               generic test optics

go.mod                  go module locations
go.sum                  go module check sums

Makefile                build scripts

favicon.ico             Swarm logo to suppress browser errors
```

MAKE

The project comes with an extensive **Makefile** to support developing.

Below are the build rules for executables and for running tests of POT JS.

Note that building and any of the following is not required for using POT JS. The relevant executable files are portable and part of the repo. After cloning or downloading a distribution of POT JS, all relevant files are immediately usable.

RULES

Run with **make <rule>**

```
build              build executables
example<n>         start server and run example <n>. n = 1-8
test               explain test modes and run node_inmem_test
web_inmem_test     test api interaction with go pot in-memory persisting, web
web_inmem_stress   stress test with go pot in-memory persisting, web
web_sim_test       extended exceptions tests w/out go pot connection, web
web_locnet_test    standard tests with a locally installed Swarm network, web
web_locnet_stress  stress test with a locally installed Swarm network, web
node_inmem_test    test api interaction with go pot in-memory persisting, node
node_sim_test      extended exceptions tests w/out go pot connection, node
node_locnet_test   test with a locally installed Swarm network, node
clean              prepare for building from scratch
distclean          prepare for commit to repository, including build
```

SUPPORT & DEBUGGING

For day-to-day developing, these commands may be helpful.

```
serve              start local http server serving current directory
stop               stop local http server. Ctrl-c does not.
mock               prepare for extended exception tests
unmock             reset for production or non-extended tests
locnet_install     install the local test network
locnet_start       start the local test network on docker
locnet_batch       buy stamp batch (free)
locnet_tests       run test suites on local test network
locnet_stop        shut local test network down
locnet_logs        show the entire log of the queen node
```

DEVELOPER NOTES

A JS API IMPLEMENTED IN GO

POT JS, while being a Javascript API to Go code, is intentionaly and necessarily programmed almost exclusively in Go. It would not work the other way around, i.e., by implementing more – or all – of its functionality in Javascript. But it is also desirable that it has no Javascript layer to speak of.

The exception, the small potjs.js script, is a dozen lines convenience code that replaces a few lines of intitialization code that one otherwise had to write for oneself, if potjs.js was not used (see examples/example2.html). This is an option, potjs.js can be left out. In effect, the WASM executable, all written in Go, and nothing written in Javascript, constitutes all that is required to use POT JS.

There is no alternative to this design. But eventually this focus on Go is not a disadvantage. The following explains why it is necessary but also desirable to have POT JS programmed almost exclusively in Go and details how that is the cleaner solution.

◊

POT JS is really Javascript without Javascipt code: even though POT JS is written in Go, what this Go code creates are Javascript objects and functions. For all intents and purposes, POT JS, at runtime, is mostly Javascript. The Go code creates Javascript runtime elements that constitute the actual API. That is, methods like kvs.put() are functions written in Go but executed by the Javascript runtime like native Javascript functions. The Go runtime's part is just to create them.

There is no way to do the opposite using syscall/js. I.e., there is no way to create native Go objects from Javascript. Syscall/js is a Go package after all, it extends Go, not Javascript. There are translators that create Go code from Javscript code, at compile time. But that is not the same as creating runtime objects for one language in the other at runtime, which is needed to create an API as tight and fast as POT JS.

That syscall/js works in this direction, Go to Javascript, is plausible, because the calls are coming from Javascript to Go. We want a gate from Javascript to WASM Go code, not the other way around. This is why it makes sense that the connecting elements of the API are created in Go – using syscall/js functions – to create Javascript objects and Javascript functions. In a pedantic sense, while the code is all Go, the API actually is fully Javascript. Just created by Go code, not Javascript code. Eventually, Javascript does not 'realize' that it is operating on 'non-native' Javascript objects that are created using Go and syscall/js.

Apart from that, there is a reason why at least some Go code is required and the mirror image to write the entire API in Javascript would not be possible: there are two typical classes of Go objects in play, employed by Go POT, as any Go project would: contexts and channels. Because they cannot be transferred over to Javascript that does not know the concept of channels (contexts are special

channels), they have to be managed on the Go side. This is where syscall/js incompleteness might be hurting. Maybe there could be a way to hand handles to channels over to Javascript. This would ver likely require manual handling of garbage collection on the Go side though to prevent the channels from being discarded as unused because the Go runtime has no visibility on the Javascript side. The overhead of keeping track of contexts and channels – either by keeping the objects, or by protecting them from premature garbage collection – necessarily has to be implemented on the Go side.

Therefore, even in its slimmest form, using syscall/js, one would always end up having at least some Go code in the mix. The API could never be all Javascript the way that it is now (almost) all Go.

This also trumps another reservation that might be raised: that it should seem much cleaner if one could avoid adding a main package to the Go implementation of POT in order to create a Javascript API, and to instead let Go POT remain untouched, as is, with no additional Go code added just for this specific API. Convincing as this view is, it is not feasible because of the way syscall/js works. It is a given that API-specific Go code has to be added. It can't work to just wrap Go POT into WASM and be done, all the rest being in Javascript. There has to be some Go code that creates the bridge; it can't be Javascript code.

And that Go code has to be part of the WASM. Thus only compiling Go POT to WASM won't work. One way around it would be to make the WASM act like a server, which would be slow, not the implementation of a Javascript API, but a general interface. And it would require adding code to Go POT just the same. But to be exact, Go POT is not changed, it is just made the main component of the API project POT JS. What is more unsatisfying is merely the question of how to organize the repos.

So, much of the objects and functions the Go code creates are Javascript. Thus, much of POT JS is really Javascript, just not Javascript code. Something must be 'the bridge' and these Go-created Javascript objects are it.

◊

Now, if the question was asked from a high level, whether having virtually NO Javascript code can be a good choice for a Javascript API, the answer is that the nature of syscall/js to mainly offer the possibility to 'program Javascript in Go', but not vice-versa, determines all else. However, even those aspects that would appear to unquestioningly lend themselves to be implemented in a complementary Javascript layer, rather than in the Go code, turn out to only work when programmed in Go, instead of in Javascript.

This is a major challenge that adds to the picture: transacting between Javascript and Go can deadlock the Go code when certain functionality is written in Javascript, instead of in Go. This even includes the creation of something as close to home as Javascript promises.

As an epitome of Javascript programming, it must seem strange that POT JS creates them in Go. However, it simply doesn't work in the way that would look more intuitive, i.e., creating a Javascript layer that defines the promises, which's executors then call into the Go WASM functionality synchronously. That this does not work, as much as it would seem that it should, can easily be confirmed by trying it out. Below is some background that may help to explain why this is so.

In the end, the answer to why POT JS is mostly programmed in Go is, it is not a choice.

But beyond these hard facts that favour coding POT JS in Go, there are reasons, given below, why it can be regarded as optimal design. The right choice, if it was not inescapable.

◊

From a high-level it is preferrable to have code only, or mostly, on one side – Javascript or Go – and to not cross the language barrier multiple times, to keep things robust and avoid unknown unknowns. Keeping things mostly on one side can minimize error sources and overhead that may occur unwittingly when switching between languages, as well as the loss of original state that this entails (channels on the Go side, exceptions for Javascript). This is why it looks preferrable, independently of the limitations that partially dictate the answer, to keep the bulk of code either on the Go or the Javascript side.

The way syscall/js works, this predisposes the Go side then.

Keeping things simple – by avoiding language switch overs – becomes more relevant for the disclaimer that comes with syscall/js, that it is "experimental and not for production." There seem to be no complains that it is not stable. It may be Google's typical traditional caution. But it encourages to keep things simple and straight forward and to avoid demanding constructs where possible. Keeping it all on the Go side then, after some code on the Go side is unavoidable in the first place, appears to be the right choice to increase robustness.

But even if all that wasn't acceptable as convincing argument for pushing everything into Go, it turns out there is a technical killer argument: as mentioned, it does not work to have asynchronicity being taken care of on the Javascript side. It does not work to have the promise wrapping for asynchronous calls be on the Javascript side. And when using WASM with node, it's not only the fetch call that is locking up – as the syscall/js documentation warns – but every synchronous call deadlocks (sic), including when using in-memory storage as persister, which is not using a fetch internally. It does not work to create the promises in Javascript for the purpose of a Javascript-side layer. They have to be implemented in Go, as POT JS is doing. Specifically, what does not work is to wrap sync calls to Go WASM in Javascript promise executors on the Javascript side.

But using 'the same' logic creating the Javascript promises on the Go side, works:

https://github.com/brainiac-five/potJavascript/blob/86fc8af3ea6b0f148bed1efb34d8dd00190bd1b3/potJavascript.go#L50

The syscall/js documentation is not clear that this would be required to avoid deadlocks. There is a description on how deadlocks should be avoided by using go routines. But it is not explained why creating the asynchronicity on the Javascript side should be less effective. Under certain conditions, this deadlock happens every time. What causes it seems to not only be the browser's main fetch thread, because it also affects node.js, and even more profoundly. With node.js, The deadlocks happen reliably unless the Javascript promises are created on the Go side. Which in fact entails a go routine to uncouple the promise executor from the synchronous program flow. Just as the syscall/js documentation seems to propose. While the Javascript asynchronicity inherent in the promise creation, empirically, does not suffice.

This might be because in a 'go <executor>' call the go routine call's mechanics are truly multithread, implemented such that if the operating system allows, go routines might run on parallel threads. Maybe this is what is preventing the whole mechanism from locking up by virtue of a stronger decoupling effected by a ago routine call. In that interpretation, the fact that doing the same on the Javascript code side – wrapping sync API calls into Javascript promises – always locks up could be explained by the fact that in Javascript nothing is ever truly parallel, as a basic strategy to keep Javascript programmers safe even while the paradigm is strongly async (except Web Workers, which add exactly that: mutlithreading). Go routines though are designed to possibly run in parallel if the OS and hardware allows for it. Conversely, this thesis is weakened by the circumstance that Go in WASM is said to always run on only one process.

Regardless, calling from Javascript into Go and then Go calling back to Javascript implicitly by making a fetch call seems to only ever work by go routine decoupling. When using node.js, there seems to be an additional cause for reliable deadlock in play when not decoupling calls with a go routine. Always building such a go routine into a Javascript promise makes for a very desirable architecture though that has no lamentable drawbacks. It even has the precious advantage that such Go-native promises provide the only way to trigger native Javascript exceptions from Go.

As the promises are the 'most outside' hull of calls though and seeing that even they cannot be implemented on the Javascript side, little else can.

But as noted, because for the sake of robustness it looks desirable to have as much code as possible on one side, and to avoid crossing language lines within one call, the necessary design also looks desirable. Even if it is surprising that eventually there is no real Javascript layer.

This is why POT JS is implemented almost entirely in Go, and why this is a desirable outcome, too.

NOTES ON DEBUGGING

Being wrapped in WASM to interface with Javascript, the Go executable is opaque. It cannot be debugged as there are no debuggers for WASM, and it becomes unavailable once it has panicked.

Consequently, developing benefits from being test-driven to unearth unexpected problems.

A wide array of tests are available in test/kvs_test.js. They include examples of failure test of varying nature.

Logging directly to terminal or browser console can be done with the Go function log(). It uses the interface itself to have direct access to Javascript's console.log.

NOTE ON SYSCALL/JS

The connection between Go and Javascript through WASM is based on Go's **syscall/js** package. This package is officially labeled as experimental by its provider, Google:

> "This package is EXPERIMENTAL. Its current scope is only to allow tests to run, but not yet to provide a comprehensive API for users. It is exempt from the Go compatibility promise."
>
> https://pkg.go.dev/syscall

This is a stark warning going beyond Google's typical caution. It may explain the incompleteness of syscall/js, e.g., the lack of an option to directly throw exceptions. What remains unclear is why this imperfect state has been going on for so long.

This also makes it look possible that, e.g., the node.js WASM crashes that report deadlocks are not in fact always deadlocks but sometimes a misfiring, overzealous deadlock detection. The Go WASM executable stops at any rate. The insight would not be the path to a solution.

INITIALIZATION

Starting out, the Go WASM executable has to be loaded, both for a HTML5 application or when using POT JS with node. In the browser this can be handle conveniently by including **potjs.js**, which also provides the wait function **pot.ready()** The core of **potjs.js** are only a few lines of code and they can alternatively be programmed directly in your source code, omitting **potjs.js**, as shown in **example2.html**. In some situations that might give helpfully more fine-grained access to design the start-up sequence (the example happens to use newSync()):

```
WebAssembly.instantiateStreaming(fetch("pot.wasm"), go.importObject)
    .then((r)=>{
        go.run(r.instance)
        map = pot.newSync()
        ...
    })
```

Generally, only after the **pot.wasm** executable has loaded – and in the course, started and is running – is the global **pot** object created and the POT JS functions described above become operational. Before that point, the **pot** object and the functions do not exist and trying to use them will cause the according JS exceptions to be thrown.

But if **potjs.js** is used, it creates the global **pot** object earlier, before **pot.wasm** is done loading and starting up, and provides through **pot.ready()** a promise that can be used to wait for **pot.wasm** to complete loading. But **pot** and **pot.ready()** are the only elements of the API that already exist at that point, no other function does until the promise of **pot.ready()** resolves.

```
await pot.ready()
map = await pot.new()
...
```

Without **potjs.js**, **onWasmLoaded()** can be created instead, which will be called by **pot.wasm** as soon as it is ready:

```
function async onWasmLoaded() {
I       map = await pot.new()
        ...
```

The **pot.wasm** executable loads quickly. Most of the time, it will not be strictly needed to use **pot.ready()** or **onWasmLoaded()**, as demonstrated by **example2.html**. The interaction coming from the user will practically always happen only after **pot.wasm** has long been loaded.

This can be a source of errors though in the case of a network hic-up, and interoperation in a node.js program will generally be too fast to ignore the need to wait for **pot.wasm** to finish loading.

TYPES

A Javascript developer will make assumptions about how types work. To provide the least surprising solution,[2] Javascript types are automatically converted to and back from the raw binary storage format values are stored in to come back as-stored.

The Go POT implementation stores raw binary data. In Javascript the equivalent would be **Uint8Array**. But that is not idiomatic to normal Javascript use. **Uint8Array**s cannot be used for keys,[3] and cannot be compared with each other as would be expected. They are generally less practical as values for a lot of use cases in Javascript, where the expectation would be that sending a variable of a certain type to a key-value store should later get you that variable in that type back, rather than some raw data that has to be type cast to be what it once was.

[2] *en.wikipedia.org/wiki/Principle_of_least_astonishment – "In user interface design and software design, [1] the **principle of least astonishment** (POLA), also known as principle of least surprise, [a] proposes that a component of a system should behave in a way that most users will expect it to behave, and therefore not astonish or surprise users."*

[3] Because they do not have a natural order in Javascript.

Javascript being weak-typed does not help, it rather heightens expectations regarding automation of type handling and on-the-fly casting.

To achieve type-awareness, type encoding has been implemented that can put and get Javascript values without losing their types. It writes a first byte to any `put` (that does not have `Raw` in its name) that reflects the JS type. This makes data stored with `put`() mostly incompatible with any get function that has `Raw` in its name.

But this is not the only way for a user to deal with types. There are *raw* access functions that deal with types on *getting*: `getBoolean()`, `getNumber()`,`getString()`. They are for reading data stored with `putRaw()`, where only one byte is stored for Booleans, 8 for numbers, etc. without the leading type byte.

This dual API for type-aware storing is separate to a third one that just stores raw `Uint8Array`s, using `putRaw()`, and then `getRaw()`. This leaves the task to the user to cast to the right type.

NUMBERS

Javascript numbers are stored to Swarm as IEEE 754 floating point standard byte code.

The reason is that the universal number type in Javascript, *even integers* (other than the special case of `BigNum`s), are internally stored as 64-bit floats. Direct access to the bits representing these numbers guarantees lossless conversion, especially in edge cases.[4]

The principally desirable representation of *integers* by their hexadecimal byte corollary was renounced as it would have meant special casing the number conversion, i.e., handling integers and floats differently depending on their face value not by any distinction Javascript itself knows or cares about. It would make some Javascript numbers into something they are not (integers), for little tangible advantage other than the hypothetical portability of POT storage between clients of different languages.

If cross-system portability of POT data access becomes a goal, then the higher robustness of an integer representation for integers would be convincing – *if* the input could reliably be known to have been meant as an integer in the first place, which in Javascript could be difficult to ascertain. Instead, an additional type marker could be added that identifies integers and allows Javascript to read them but not write them.

BIG NUMBERS

Javascript Big Numbers could not be implemented at this point because the Go `syscall/js` package – being officially incomplete, see above – can currently not handle them. It seems likely that `syscall/js` could be extended to make `BigNum`s work if desired. There are receipts for it online.

[4] The alternative of storing numbers as strings works well, too, but goes through a much more involved process of conversion that can be assumed to be much more time and memory-consuming. While in the light of this context this does not matter too much, it might also be more prone to errors in edge cases such as the likes of `10/3`, `-0`, and *multiple* variations of `NaN` in IEEE 754.

While modding the standard Go package for WASM communication with JS opens a can of worms, it seems important enough eventually to allow for `BigNums` to be stored in Swarm natively.

TESTS

To accommodate for the dual origination of errors – on the JS or Go side – the combined test suite utilizes the same JS ↔ WASM ↔ Go channel that the API itself uses to trigger, evaluate and log the tests, jostling between languages. A number of functions were added to enable black box testing. They are no ops in production, neutered via Go build directives.

To be able to test its reaction to delays, stalls, exceptions and cancellations, and to insulate tests of the JS API from any potential problems with the Go POT implementation itself, which is not yet battle-hardened or extensively tested, the 'extended' tests implement their own mock back end (`mock.go`). This is a second in-memory storage variant, separate from the in-memory storage the Go POT implementation itself offers per default and that is used in the 'standard' tests.

This mock back end, besides being helpful for debugging, exists to emulate fringe conditions that could not otherwise be tested but must be expected when the Go POT package operates connected to the Swarm network. E.g., mock delays are essential to test the basic user cancellation and time out design that has to satisfy both Javascript and Go language idiosyncrasies to prevent resource leaks in Go from dangling sub routines that were not freed when a request did not succeed. As a special complication, both runtimes have their own garbage collector, which always add opaqueness but, in the instance, have to be made to work together in their estimation of what resource can be discarded. In this regard, Go channels are a special challenge, as well as a potential source of deadlocks.

ERROR HANDLING

The divergent error handling philosophies of Javascript and Go had to be balanced. A Javascript programmer does not expect errors coming back as return values – like in Go – but as exceptions. But Go cannot trigger Javascript exceptions through WASM. It can `return` a value of type `Error` as JS-native `Error` type return value but not `throw` it directly, due to the incompleteness it seems of `syscall/js`.

While `throw` is not available to Go, a middle ground is provided by JS promises that enable – when created in Go instead of in JS – by the use of their `reject` function a means to trigger an exception in Javascript. This constitutes an additional reason for why the API functions that return promises are preferrable. It allows for the most seamless transportation of errors from inside Go to Javascript in the expected way for a JS programmer. And calls from JS to WASM, one can read, should generally

be promises as the nature of WASM is not quite like a library without agency but more like a parallel process. (It is not as it is executed on the same thread as the JS main thread).

Synchronous calls cannot benefit from this way of throwing errors. They are made to return a Javascript Error object instead of the expected return value. This does not fit well in the Javascript workflow and there are possible solutions. A promise can be created on the Go side in case of an error that immediately rejects. However, this still leads to asynchronicity, literally, between the error and the arrival of the error message, and "uncaught (in promise)" type errors that arrive a little bit late. E.g., often after a test program has finished.

A second solution is described in
`https://stackoverflow.com/questions/67437284/how-to-throw-js-error-from-go-web-assembly`:
the WASM compiler can be extended using a hint, and the generic `wasm_exec.js` enhanced to add the possibility to `throw` from Go.

Because this seems only interesting for synchronous functions, which are not central to the overall user experience, the effort and the risk of touching the compiler and `wasm_exec.js` – where until now they are used without modification – seem not appropriate. The less than optimal solution will remain that synchronous functions return error objects when things go wrong, and that async functions that return promises should be preferred.

SLOTS

The array of `Slot` structures, `slots`, keeps the state of a KVS on the Go side that cannot be transferred to Javascript. On first glance, one would prefer a different, more elegant solution. But on inspection, they might be the best solution. Eventually, in what is a bit of a mine field, they are sufficiently simple that they can be made robust.

The slots hold the Go objects and channels for which how to create a safe JS handle for is not obvious and would create garbage collection issues. A handle for a Go object that could, e.g., be created from its memory address and given to JS, might have the effect that Go destroys the object if it detects that it has become unreachable by Go's count. Go's `runtime.keepAlive()` is designed to work within one function. Maybe a strategically created closure could keep variables alive within it and could prevent the collection. But that starts to look error prone and no better than a straight array of state.

Another thinkable alternative to the `slots` array could be to use closures to hold the state itself even where it cannot be transferred to Javascript and attach those closures to the Javascript object that is created for a new KVS. This works for the Go context `cancel()` functions of the JS promises. But while a Go function can be wrapped in a `js.Func` – a Javascript function that is created in Go and visible on both sides – and this closure could then be called by calling the wrapping JS functions from within Go (instead of from JS as they are intended), state cannot be accessed from Go when wrapped in this way because you don't get it back out. The return value of a js.Func must be a JS object. This

cannot return anything that is not convertible to JS, like channels are not. It is visible in the `syscall/js` package's source code that the implicit, automatic conversion from Go to JS values cannot help to cross this threshold as it has the same limitations. `syscall/js` could probably be modified to circumvent this block but that would have to include the task to make sure that the garbage collector stays up to date and starts to look too involved.

A yet more highwire solution could be found by enclosing Go variables in the `js.Func` closure that can then be accessed through a wrapping Go function because the variable exists in both the inner and outer closure. But the outer closure itself cannot be attached to the JS object and would still have to be held somehow on the Go side – e.g., in an array. One would again resort to a map or an array to hold the access at hand to the relevant original Go objects. There seems to be no way to transfer state alien to JS to JS and get it back out to Go. That's by design.

This is why a state array like `slots` is useful, even while the `cancel()` functions of the promises work without such a central list. The Go state is simply kept on the Go side.

The integer index of a `Slot` in the `slots` array + 1 is used as the handle that is given to the Javascript KVS object as a Javascript number. `this.slot_ref` on the JS side holds it. A number can be passed to Javascript without a problem – even if there is the theoretical caveat that by necessity the integer is converted to a float, as JS does not have native integers, except for `BigNums` that `syscall/js` cannot handle. Thus, theoretically a conversion to a string would be safer. But in practice, JS using 64 bit IEEE 754 floats, the first integer conversion to fail is 9,007,199,254,740,993.

The `Slot` struct is retrieved on every call to `get()`, `put()` etc. accessed through the `this` pointer that is the first parameter that every `js.Func`-wrapped Go function receives when it is called from JS. The slot index `+ 1` is the number that is stored in the `slot_ref` field of the Javascript KVS object. This is what is accessed through the `this` reference in a `put()` or `get()` call. `this.slot_ref` is used as index `+ 1` to access the right Go `Slot` struct of the KVS that is being operated on from the slots array. The +1 keeps 0 an invalid `slot_ref` as a safety measure.

The most important element in the `Slot` struct is the Go `SwarmKvs` pointer to the struct that is returned from `newSwamKvs()`. It holds, via Go `pot.SwarmKvs`, `pot.Index` that is the struct holding the channels that are required by `muxProcess()`, the function that loops through a `select` that sequences write access to a POT. Every KVS has its own. It kicks in when it is created with `pot.new()`.

ALTERNATIVES

It can seem that a different locking mechanism than `muxProcess()`'s `select` could be implemented on the Go side as Javascript's single-threaded nature could be taken to represent a free replacement for a multiplexer. Hypothetically, `muxProcess()` was not needed in its current form when interfacing to Go POT from Javascript, neither from the browser nor from node.js unless Worker threads were employed; by the logic that Javascript is already single-threaded and needs no protection against parallel writing. One might even ask whether the doubling locking aspect of this doubling of sequencing (that JS because it is single-threaded by nature does nothing in parallel ever, which muxProcess() doubly makes sure does not happen for writes) might be what unintentionally causes the not quite explained deadlocks of *all* sync calls coming from node.js. Perhaps the channels and

`muxProcess()`'s constantly waiting `select` might not be required, the channels could possibly be left out altogether or created per-call instead so that they do not have to be kept available between calls – in the `slots` array – with the KVS object. This could be a required modification to make JS and Go play together more nicely.

If this would work, it would leave only the second type of channels to deal with, the `contexts`. They could likewise be created per-call so that they would not have to be kept around between calls, at the cost of losing the possibility of a KVS-wide cancellation option.

The last relevant object held in Slots then would be the persister. As this is not currently keeping a connection alive it could in its production variant – that connects to a Bee node via http – be implemented in a form where its state could be transferred to Javascript, if it needs state at all. Alternatively, the persister could be architected to be a native Javascript object, implemented in JS (code) in the first place. The bigger obstacle in this case would be the special, state-holding, in-memory version of the persister that has to be kept alive between incoming calls from JS to not lose its state. But the overall architecture should not be bent to make this less-important, in-memory variant of the persister easy to implement. To the degree it is practical for testing, the persister code itself should be the place where the kludge should be residing that makes state-holding between calls possible; rather than burdening the overall architecture.

But even for such streamlined version of Go POT and POT JS where Go POT is altered to work better with POT JS, `muxProcess()` might turn out to not be superfluous. It is already daring to rely on Javascript to never really do things in parallel. That means banking on that there will never be two write calls overlapping each other coming from JS in a way that would trip up Go POT. Apart from the fact that some future browser optimization might break guarantees that are currently implicit but not spelled out. A browser might do something 'helpful' that ends up using more than one thread to speed JS up and by that break the single-thread assumption. This is not unrealistic because any measures such improvement would require building into the JS runtime would be invisible and unavailable when control is given over to WASM.

But even as is, JS' asynchronicity could be enough to have two write promise's executors end up switching back and forth in a way that they look parallel to the Go side. In the end the promises are there to make things look like they happen in parallel and have multiple things pending. That's exactly what `muxProcess()` is there to prevent for writes. Multiple writes can pend but they must not happen at the same time, and this sequencing is what `muxProcess()` provides. JS would push things up to the point where it looks like the writes are both happening and then wait for the result, it does not nicely queue them.

Finally though, Go compiled to WASM does itself only run on one thread and cannot use its potential to multithread. It is not the case that a speed up through parallelizing and multiplexing could be expected, with parallel writes for different KVS, even while parallel writes for a single KVS is what `muxProcess()` exists to prevent. It is also not in this constellation the case that it is a useful capacity of Go that JS can't deal with and thus it has some high-level consistency that it has to park the controlling state of this Go superpower outside. Go itself can when compiled to WASM not do what `muxProcess()` is designed to prevent it from doing in the wrong moment: work things in parallel. It would be plausible to expect that `muxProcess()` at least prevents that two writes race each other by switching back and forth between the two in an unlucky way. However, it would be enough to not use go routines to make sure that no two writes could race each other because WASM is executed

on a single thread by Javascript. The expectation has to be that `muxProcess()` is unneccesary when Go POT is used in WASM. Unlike its use in a different environment, e.g., when used in Go as a library or package of a bigger system, e.g., included in a Bee client, there is no need to take care of the case that `put()` calls could be coming in even on different green threads, much less truly parallelly.

Together with surprises and limitations that surfaced while joining Javascript and Go, it looks difficult to judge whether an attempt at improving this would be worth the try.

SLOTS MIRROR A POWER IMBALANCE

In the end, in a holistic view, the core of the problem is in the first place that the sequencer state – the `muxProcess()` channels – would ideally be carried over from Go to Javascript to be held there, which we can't and thus have the `slots` array for; while what's technically only needed, instead, would be to either make sure there is no asynchronicity started within Go POT during the handling of one `put()`, or to have a simple semaphore per KVS on the JS side that makes sure that no two write calls are triggered from JS at the same time. In the latter case, there could still be multiple promises for puts created and pending at about the same time, but they would simply have to prevented from running their executors to the blocking point of a write at the same time, and not rely on asynchronicity alone that would avoid blocking for JS but not a write race on the Go side. Abstractly, this would seem to not change much, because at present, that queuing is happening in `muxProcess()` on the Go side. But it would be a simpler model, more robust and possibly apt to avoid the deadlock crashes[5] of the Go WASM executable that might be false positive of an overly sharp deadlock detector of the Go runtime (that was historically too lenient).

But overall, making Go POT cater to JS's needs would be an optimization the time of which probably has not come. In light of all of the above, the simplistic slots array looks like the best solution. Its inelegant ways are somewhat explained by how it is an artefact of the overall architecture not being optimal from the JS perspective. With its possibility for true parallel execution, Go POT is more powerful than what JS can make good use of having it in WASM and figuratively speaking, that power needs to be parked somewhere, unused, in the form of the channels.

Go POT on the other hand is meant to have the capability to work truly in parallel, because on the one hand the index updates could with big tries tilt from network-bound into processing-bound. I.e., the wait-time could be caused by the calculations happening in Go POT proper rather than at all times be the wait for an answer back from the Bee API. On the other hand, how it is crafted is just the go way of implementing asynchronous calls to the storage network.

In this view, the construct of the `slots` array is an artifact that betrays a friction across the overall implementation, joining Go and JS, and stemming necessarily from that Go POT was not made as a JS backend and can do more than JS can ask of it, namely work in a truly parallel fashion. The `slots` array deals in exactly those Go elements that JS cannot hold, i.e., the channels, for something it was not designed to do, real multi-threading. And as a consistent consequence it does not have the means to hold the special type of state either that is native to Go to make true multithreading possible.

[5] To be clear: that happen preditable and reliably, not

ROADMAP

Possible future improvements:

- FUNCTION TIMEOUTS
- PROOF RETRIEVAL FUNCTION
- TREE WALK FUNCTIONS
- GO-SIDE RESOURCE TRACKING INSTRUMENTATION
- JS-NATIVE PERSISTERS
- USE OF JS NEW OPERATOR
- INTEGRATION TESTS